

Documentación Técnica

LogísTEC

Segundo Proyecto — Algoritmos y Estructuras de Datos I (CE1103)

Tecnológico de Costa Rica

Lenguaje: Java 17 o superior

Formato de entrada: JSON

Integrantes:

Brandon Ezquivel

Sebastián Leiva

Jeremías Cubero

Issac Arce

Fecha: 12/6/26

a) Manual de usuario

Descripción general

LogísTEC es una aplicación desarrollada en Java para resolver un problema de planificación logística sobre grafos. La ciudad se modela como un grafo no dirigido y ponderado, donde los vértices representan intersecciones, puntos de entrega o el depósito principal, y las aristas representan calles bidireccionales con una distancia en metros.

El sistema carga un archivo JSON con la ciudad, paquetes y camiones. Luego valida la conectividad, calcula caminos mínimos, construye árboles de expansión mínima, asigna paquetes a camiones y genera rutas de entrega con dos heurísticas: Nearest Neighbor y MST-based.

Requerimientos de hardware

Recurso	Requerimiento recomendado
Procesador	Intel Core i3 / Ryzen 3 o superior
Memoria RAM	4 GB mínimo, 8 GB recomendado
Almacenamiento	100 MB libres para código, librerías y archivos de prueba
Pantalla	Resolución mínima 1366 x 768 para visualizar la interfaz gráfica

Requerimientos de software

Software	Versión recomendada	Uso
Java JDK	17 o superior	Compilar y ejecutar el proyecto
Gson	2.10.1	Parseo del JSON de entrada
Sistema operativo	Windows	Entorno de ejecución
Editor recomendado	VS Code, IntelliJ IDEA o NetBeans	Edición y revisión del código

Estructura esperada

```
LogisTEC
├── data
│   ├── caso_pequeno.json
│   ├── caso_mediano.json
│   └── caso_grande.json
├── docs
├── lib
│   └── gson-2.10.1.jar
├── src
│   ├── algorithms
│   ├── ds
│   ├── graph
│   ├── io
│   ├── model
│   ├── planner
│   ├── ui
│   └── Main.java
├── compilar.bat
├── ejecutar.bat
└── compilar_y_ejecutar.bat
```

Compilación y ejecución

En Windows, desde la raíz del proyecto, se puede compilar con:

```
compilar.bat
```

También se puede compilar manualmente con:

```
dir /s /b src\*.java > sources.txt
javac -encoding UTF-8 -cp "lib\gson-2.10.1.jar" -d out @sources.txt
```

Después de compilar, se ejecuta con:

```
ejecutar.bat
```

Para ejecutar manualmente una instancia específica:

```
java -cp "out;lib\gson-2.10.1.jar" Main data\caso_grande.json
```

Formato esperado del JSON

El archivo JSON debe contener una ciudad con vértices y aristas, además de paquetes y camiones. Los vértices deben tener identificador, tipo y coordenadas; las aristas deben tener dos extremos y una distancia; los paquetes deben tener destino, peso y prioridad; y los camiones deben tener capacidad.

```
{
  "ciudad": {
    "vertices": [
      {"id": "V00", "tipo": "DEPOT", "x": 80, "y": 80}
    ],
    "aristas": [
      {"u": "V00", "v": "V01", "distancia": 140}
    ]
  },
  "paquetes": [
    {"id": "P01", "destino": "V03", "peso": 6, "prioridad": 1}
  ],
  "camiones": [
    {"id": "C01", "capacidad": 25}
  ]
}
```

Tipos de vértice aceptados

Tipo lógico	Valores aceptados en JSON
Depósito	DEPOT, DEPOSITO, DEPÓSITO
Entrega	DELIVERY, ENTREGA
Intersección	INTERSECCION, INTERSECCIÓN, INTERSECTION

Salida del programa

La consola muestra el resumen del caso, validación con Warshall, camino mínimo con Dijkstra, matriz Floyd-Warshall, comparación de MST, asignación de paquetes y rutas de camiones. Además, se abre una ventana Swing con el grafo y las rutas.

b) Bibliotecas de terceros utilizadas y propósito

Biblioteca	Versión	Propósito	Justificación
Gson	2.10.1	Lectura y conversión del archivo JSON a objetos Java	El parseo de JSON no es el objetivo central del curso, por lo que se usa una biblioteca externa permitida.
Swing	Java SE	Construcción de la interfaz gráfica	Forma parte de Java estándar y no requiere configuración adicional.
AWT	Java SE	Dibujo de vértices, aristas, rutas y colores	Permite renderizar la visualización del grafo dentro de un JPanel.

c) Métodos implementados por el equipo, organizados por paquete

Paquete graph

Clase	Métodos principales	Propósito
Graph	addVertex, addEdge, getVertex, getVertexByIndex, getVertices, getEdges, getNeighbors, vertexCount, edgeCount	Modela la ciudad como grafo no dirigido ponderado.
Vertex	getId, getType, getX, getY, getIndex	Representa intersecciones, entregas y depósito.
Edge	getU, getV, getDistance, getOther	Representa calles bidireccionales con distancia.
VertexType	fromString	Convierte tipos textuales del JSON a tipos internos.

Paquete ds

Clase	Métodos principales	Propósito
DLinkedList	addLast, addFirst, removeFirst, contains, size, isEmpty, toArray	Lista doblemente enlazada genérica.
Queue	enqueue, dequeue, isEmpty, size	Cola FIFO usada en BFS.
Stack	push, pop, isEmpty, size	Pila LIFO usada en DFS.
MinHeap	insertOrDecrease, delMin, extractMin, isEmpty	Cola de prioridad usada por Dijkstra y Prim.
UnionFind	find, union, connected	Conjuntos disjuntos usados por Kruskal.

Paquete algorithms

Clase	Métodos principales	Propósito
Traversals	bfs, dfsIterative, dfsRecursive, connectedComponents, isConnected, isReachable	Recorridos y conectividad.
Warshall	compute, unreachableFromDepot	Cierre transitivo y paquetes inalcanzables.
Dijkstra	distTo, hasPathTo, pathTo, allDistances	Camino mínimo desde un origen.
FloydWarshall	dist, hasPath, path, getDistMatrix	Caminos mínimos entre todos los pares.
Prim	totalCost, elapsedNanos, edges	MST mediante crecimiento por vértices.
Kruskal	totalCost, elapsedNanos, edges	MST mediante ordenamiento de aristas y Union-Find.

Paquete io

Clase	Métodos principales	Propósito
JsonLoader	load, loadFromString	Carga y valida el JSON.
LogisticsConfig	depotId, vertices, edges, packages, trucks	Estructura intermedia con la configuración parseada.
ConfigValidationException	constructores	Reporta errores de configuración.

Paquete model

Clase	Métodos principales	Propósito
ProblemCase	from, getGrafo, getPaquetes, getCamiones, getDepot	Representa el caso logístico completo.
Package	getId, getDestinoId, getPeso, getPrioridad, isRechazado, rechazar	Representa paquetes y su estado.
Truck	getId, getCapacidad	Representa camiones y su capacidad.

Paquete planner

Clase	Métodos principales	Propósito
PackageAssigner	assign	Asigna paquetes a camiones.
AssignmentResult	getAssignments, addRejectedPackage, printSummary	Guarda el resultado global de asignación.
TruckAssignment	canFit, addPackage, getDestinationIds, getOccupationPercentage	Guarda los paquetes de un camión.
NearestNeighborRouter	buildRoute	Calcula ruta por vecino más cercano.
MSTBasedRouter	buildRoute	Calcula ruta basada en MST.
RouteResult	getHeuristicName, getRoute, getTotalDistance	Guarda una ruta calculada.
RouteComparison	getBestRoute, getImprovementPercentage	Compara dos rutas.

Paquete ui

Clase	Métodos principales	Propósito
LogisTecFrame	showWindow	Crea la ventana principal.
GraphPanel	setSelectedRoute, paintComponent	Dibuja grafo, rutas y leyenda.

d) Estructuras de datos desarrolladas con API y complejidad

Lista doblemente enlazada — DLinkedList<T>

Estructura lineal donde cada nodo conoce al anterior y al siguiente. Se usa para almacenar vértices, aristas, paquetes, camiones y recorridos.

Operación	Complejidad
addFirst	$O(1)$
addLast	$O(1)$
removeFirst	$O(1)$
contains	$O(n)$
size	$O(1)$
isEmpty	$O(1)$
toArray	$O(n)$

Cola — Queue<T>

Estructura FIFO usada principalmente en BFS.

Operación	Complejidad
enqueue	$O(1)$
dequeue	$O(1)$
isEmpty	$O(1)$
size	$O(1)$

Pila — Stack<T>

Estructura LIFO usada principalmente en DFS iterativo.

Operación	Complejidad
push	$O(1)$
pop	$O(1)$
isEmpty	$O(1)$
size	$O(1)$

Cola de prioridad — MinHeap

Heap mínimo sobre arreglos. Permite extraer el elemento con menor prioridad y actualizar prioridades.

Operación	Complejidad
insertOrDecrease	$O(\log n)$
delMin / extractMin	$O(\log n)$
isEmpty	$O(1)$
contains	$O(1)$ si mantiene posición

Union-Find

Estructura de conjuntos disjuntos usada en Kruskal para evitar ciclos.

Operación	Complejidad
find	$O(\alpha(n))$
union	$O(\alpha(n))$
connected	$O(\alpha(n))$

Grafo no dirigido ponderado — Graph

Representa la ciudad, sus intersecciones y calles.

Operación	Complejidad
addVertex	$O(1)$ u $O(n)$, según validación
addEdge	$O(1)$ si recibe vértices ya validados
getVertex(id)	$O(n)$ si busca linealmente
getNeighbors	$O(\text{grado}(v))$ u $O(E)$, según almacenamiento
vertexCount / edgeCount	$O(1)$

e) Algoritmos implementados con pseudocódigo y ejemplo trabajado

Nota: Para los ejemplos se usa un grafo pequeño con vértices A, B, C y D. Aristas: A-B=2, A-C=5, B-C=1, B-D=4 y C-D=2.

BFS

Propósito: Recorrer el grafo por niveles desde un vértice inicial.

Complejidad: $O(V + E)$

Pseudocódigo:

```
BFS(G, origen):
    crear cola Q
    marcar origen como visitado
    encolar origen
    mientras Q no esté vacía:
        u = desencolar Q
        visitar u
        para cada vecino v de u:
            si v no ha sido visitado:
                marcar v como visitado
                encolar v
```

Ejemplo trabajado: Desde A se visita A, luego se descubren B y C; desde B se descubre D. Un orden posible es A, B, C, D.

DFS

Propósito: Recorrer el grafo profundizando antes de retroceder.

Complejidad: $O(V + E)$

Pseudocódigo:

```
DFS(G, origen):
    marcar origen como visitado
    visitar origen
    para cada vecino v de origen:
        si v no ha sido visitado:
            DFS(G, v)
```

Ejemplo trabajado: Desde A se puede avanzar a B, luego a C y luego a D. Un orden posible es A, B, C, D.

Warshall

Propósito: Calcular cierre transitivo para conocer alcanzabilidad entre pares de vértices.

Complejidad: $O(V^3)$

Pseudocódigo:

```
Warshall(P):  
  para k = 0 hasta n-1:  
    para i = 0 hasta n-1:  
      para j = 0 hasta n-1:  
        P[i][j] = P[i][j] OR (P[i][k] AND P[k][j])
```

Ejemplo trabajado: En el grafo ejemplo, todos los vértices terminan siendo alcanzables entre sí porque el grafo es conexo. Por lo tanto, la matriz final queda llena de 1.

Dijkstra

Propósito: Encontrar caminos mínimos desde un origen con pesos no negativos.

Complejidad: $O((V + E) \log V)$

Pseudocódigo:

```
Dijkstra(G, origen):  
  para cada vértice v:  
    dist[v] = infinito  
    prev[v] = nulo  
  dist[origen] = 0  
  insertar origen en cola de prioridad  
  mientras la cola no esté vacía:  
    u = extraer mínimo  
    para cada arista u-v:  
      nueva = dist[u] + peso(u,v)  
      si nueva < dist[v]:  
        dist[v] = nueva  
        prev[v] = u  
      actualizar prioridad de v
```

Ejemplo trabajado: Desde A: distancias finales A=0, B=2, C=3 y D=5. Camino mínimo A-D: $A \rightarrow B \rightarrow C \rightarrow D$, distancia 5.

Floyd-Warshall

Propósito: Calcular caminos mínimos entre todos los pares.

Complejidad: $O(V^3)$

Pseudocódigo:

```
FloydWarshall(D):  
  para k = 0 hasta n-1:  
    para i = 0 hasta n-1:  
      para j = 0 hasta n-1:  
        si D[i][k] + D[k][j] < D[i][j]:  
          D[i][j] = D[i][k] + D[k][j]
```

Ejemplo trabajado: La distancia A-C mejora de 5 a 3 pasando por B; la distancia A-D mejora a 5 pasando por B y C.

Prim

Propósito: Construir MST agregando la arista mínima que conecta el árbol con un vértice fuera.

Complejidad: $O((V + E) \log V)$

Pseudocódigo:

```

Prim(G, inicio):
  para cada vértice v:
    key[v] = infinito
    parent[v] = nulo
  key[inicio] = 0
  mientras existan vértices fuera del MST:
    u = extraer vértice con menor key
    agregar u al MST
    para cada vecino v de u:
      si v no está en MST y peso(u,v) < key[v]:
        key[v] = peso(u,v)
        parent[v] = u

```

Ejemplo trabajado: Desde A se escogen A-B=2, B-C=1 y C-D=2. Costo total del MST: 5.

Kruskal

Propósito: Construir MST ordenando aristas y agregando solo las que no formen ciclo.

Complejidad: $O(E \log E)$

Pseudocódigo:

```

Kruskal(G):
  ordenar aristas por peso ascendente
  crear UnionFind
  para cada arista (u,v):
    si u y v no están conectados:
      agregar arista al MST
      union(u,v)
  si MST tiene V-1 aristas:
    terminar

```

Ejemplo trabajado: Aristas ordenadas: B-C=1, A-B=2, C-D=2, B-D=4, A-C=5. Se agregan B-C, A-B y C-D. Costo total: 5.

Nearest Neighbor

Propósito: Construir una ruta visitando siempre la parada no visitada más cercana.

Complejidad: $O(n^2)$

Pseudocódigo:

```

NearestNeighbor(deposito, paradas, D):
  ruta = [deposito]
  actual = deposito
  mientras existan paradas no visitadas:
    elegir parada p con menor D[actual][p]
    agregar p a ruta
    marcar p como visitada
    actual = p
  agregar deposito al final
  retornar ruta

```

Ejemplo trabajado: Con depósito A y paradas B, C, D: $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$. Distancia: $2 + 1 + 2 + 5 = 10$.

MST-based

Propósito: Construir una ruta aproximada usando un MST sobre depósito y paradas, luego DFS preorden.

Complejidad: $O(n^2)$ en la implementación sobre matriz

Pseudocódigo:


```

MSTBased(deposito, paradas, D):
    puntos = deposito + paradas
    construir MST usando distancias D
    hacer DFS preorden desde deposito
    ruta = orden del DFS
    agregar deposito al final
    retornar ruta

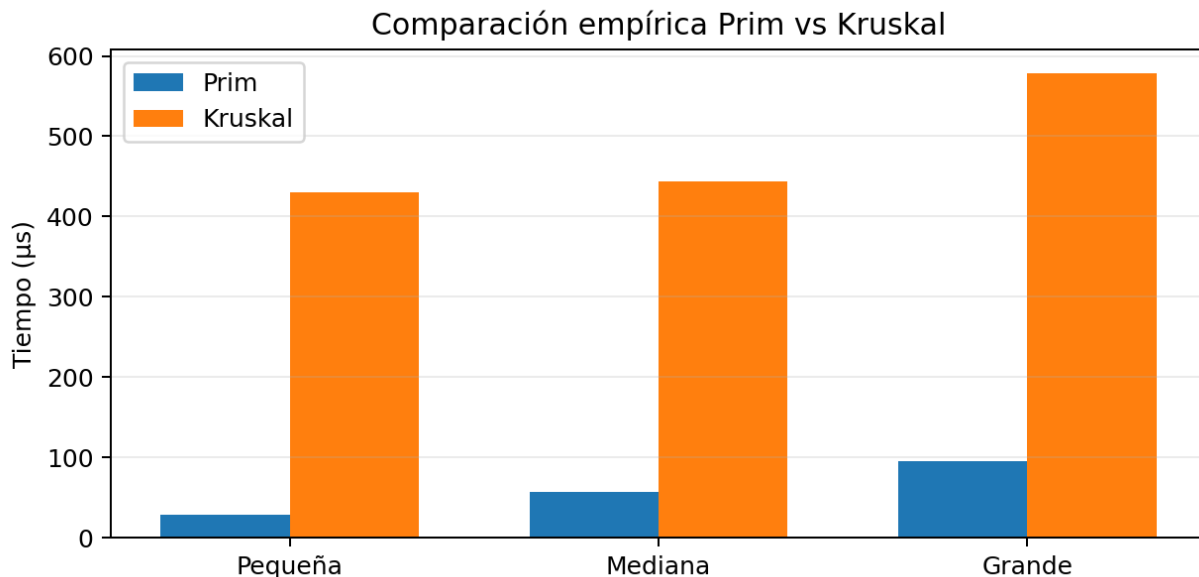
```

Ejemplo trabajado: Con puntos A, B, C, D, el MST usa B-C=1, A-B=2 y C-D=2. El DFS desde A produce A → B → C → D → A.

f) Comparación empírica Prim vs Kruskal

Se ejecutaron tres instancias de tamaños distintos: pequeña, mediana y grande. En todos los casos el grafo fue conexo y ambos algoritmos produjeron el mismo costo total de MST, lo cual valida la consistencia del resultado.

Instancia	Vértices	Aristas	Tiempo Prim (μs)	Tiempo Kruskal (μs)	Costo Prim	Costo Kruskal
Pequeña	10	15	29	430	1490	1490
Mediana	20	35	57	444	2830	2830
Grande	34	56	95	579	5239	5239



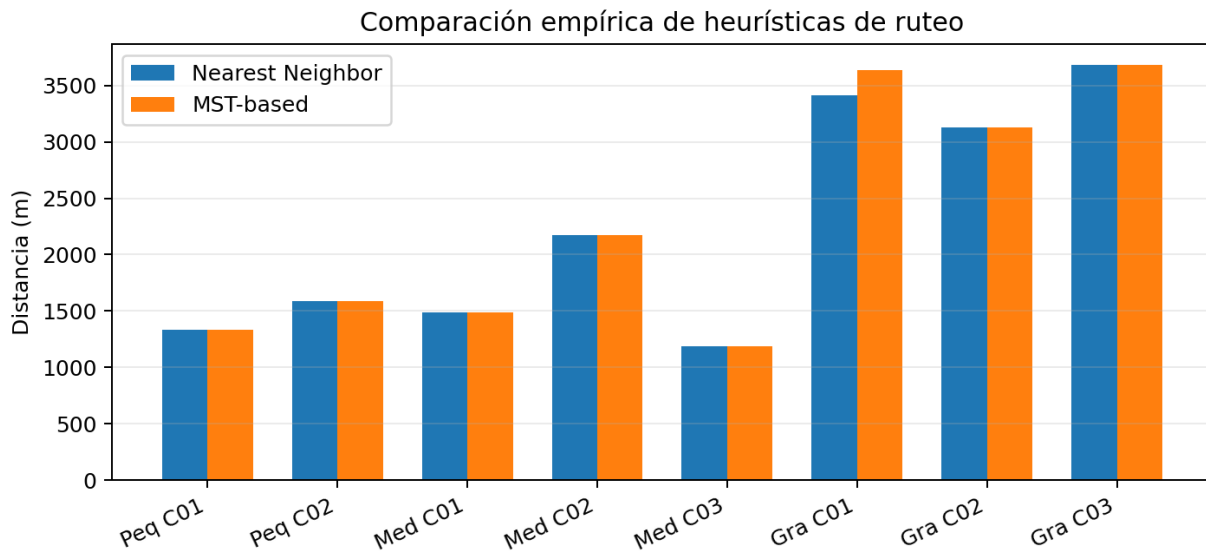
Análisis: Prim fue más rápido en las tres pruebas realizadas. Kruskal produjo el mismo costo total, pero tardó más debido al procesamiento de aristas y al ordenamiento previo. Aun así, ambos algoritmos son correctos porque el costo del MST coincide en cada instancia.

g) Comparación empírica Nearest Neighbor vs MST-based

Se compararon las dos heurísticas de ruteo para cada camión con paquetes asignados. La mejora MST sobre NN se calcula como $((\text{distancia NN} - \text{distancia MST}) / \text{distancia NN}) \times 100$. Si el valor es negativo, significa que MST-based fue peor que Nearest Neighbor en esa instancia.

Instancia	Camión	Distancia NN	Distancia MST-based	Mejor heurística	Mejora MST sobre NN
Pequeña	C01	1330	1330	MST-based	0,00%
Pequeña	C02	1590	1590	MST-based	0,00%
Mediana	C01	1485	1485	MST-based	0,00%
Mediana	C02	2170	2170	MST-based	0,00%
Mediana	C03	1190	1190	MST-based	0,00%

Grande	C01	3416	3634	Nearest Neighbor	-6,38%
Grande	C02	3126	3126	MST-based	0,00%
Grande	C03	3682	3682	MST-based	0,00%



Análisis: En la mayoría de rutas ambas heurísticas produjeron la misma distancia. Esto puede ocurrir cuando las paradas quedan distribuidas de forma lineal o cuando el recorrido generado por el MST coincide con el vecino más cercano. En el caso grande para C01, Nearest Neighbor fue mejor, ya que MST-based generó una distancia de 3634 m contra 3416 m. Esto confirma que las heurísticas no garantizan siempre el óptimo y que conviene comparar sus resultados antes de seleccionar la ruta final.

h) Problemas conocidos y limitaciones

- La interfaz gráfica es básica y está orientada a la defensa del proyecto.
- Las rutas se dibujan como líneas directas entre paradas, no como el camino completo por todos los vértices intermedios.
- Las heurísticas Nearest Neighbor y MST-based no garantizan siempre la ruta óptima.
- Los tiempos empíricos pueden variar según la computadora donde se ejecuten las pruebas.
- El sistema depende de que el archivo JSON esté correctamente formado.
- No se pueden editar datos desde la interfaz gráfica.
- La asignación rechaza paquetes cuando no queda capacidad suficiente en ningún camión.

i) Actividades por estudiante (timesheet)

Fecha	Estudiante	Actividad realizada
31/5	Brandon	Creación y estructuración de carpetas
3/6	Sebastián	Algoritmo Dijkstra
4/6	Sebastián	Agregar algoritmo Floyd-Warshall todos los pares
5/6	Sebastián	Agregar algoritmo Floyd-Warshall todos los pares
6/6	Sebastián	Agregar algoritmo Kruskal para MST, algoritmo Prim para MST
11/6	Isaac	Agrego clases del grafo (Vertex, Edge, Graph, VertexType). BFS, DFS

		y Warshal. pruebas de grafo, recorridos y Warshall y contenido a clases del grafo, recorridos y pruebas
11/6	Brandon	Tests de estructuras. Correcciones menores
11/6 – 12/6	Jeremias	Implementación de planner, rutas, interfaz gráfica y documentación

j) Problemas encontrados durante el desarrollo

Falta de la biblioteca Gson

Descripción: El proyecto no compilaba porque Java no encontraba el paquete com.google.gson.

Intentos realizados: Se revisaron los imports y se verificó si la librería estaba incluida.

Solución aplicada: Se agregó gson-2.10.1.jar en la carpeta lib y se actualizó el comando de compilación y ejecución con -cp.

Conclusión: Las librerías externas deben incluirse tanto al compilar como al ejecutar.

Bibliografía consultada: Documentación oficial de Gson y documentación de javac.

Tipo de vértice ENTREGA no reconocido

Descripción: El JSON usaba el valor ENTREGA, pero el enum VertexType no lo aceptaba.

Intentos realizados: Se revisó el método fromString y se comparó contra los valores usados por el JSON.

Solución aplicada: Se agregaron variantes en español e inglés: ENTREGA, DEPOSITO, DEPÓSITO, INTERSECCION e INTERSECCIÓN.

Conclusión: Aceptar variantes reduce errores de entrada y mejora la robustez del cargador.

Bibliografía consultada: Documentación del proyecto y revisión del formato JSON.

Diferencia entre extractMin y delMin

Descripción: Algunos algoritmos esperaban extractMin, pero el heap tenía delMin.

Intentos realizados: Se revisaron Dijkstra, Prim y MinHeap para encontrar la incompatibilidad.

Solución aplicada: Se agregó extractMin como alias de delMin.

Conclusión: Los nombres de operaciones deben ser consistentes entre TDAs y algoritmos.

Bibliografía consultada: Material del curso sobre colas de prioridad.

Interfaz gráfica saturada

Descripción: La primera versión de la interfaz mostraba todas las rutas, pesos y aristas al mismo tiempo.

Intentos realizados: Se revisó visualmente la salida y se detectó que era difícil de interpretar.

Solución aplicada: Se rediseñó la UI para mostrar grafo limpio, rutas individuales o todas las rutas mediante botones.

Conclusión: En visualización, más información no siempre significa mejor comprensión.

Bibliografía consultada: Documentación de Swing y AWT.

k) Conclusiones y recomendaciones

Conclusiones

- Los grafos permiten representar de forma natural una ciudad con intersecciones, calles y distancias.
- Warshall permitió validar alcanzabilidad antes de asignar paquetes, evitando rutas imposibles.
- Dijkstra permitió obtener caminos mínimos desde un origen y validar resultados puntuales.
- Floyd-Warshall fue útil como preprocesamiento para consultar distancias mínimas entre paradas durante el ruteo.
- Prim y Kruskal produjeron el mismo costo de MST en las tres instancias, validando la implementación.
- Las estructuras propias MinHeap y UnionFind fueron esenciales para implementar algoritmos eficientes.
- Las heurísticas de ruteo permiten obtener soluciones razonables sin resolver TSP de forma exacta.
- La comparación de heurísticas es necesaria porque MST-based no siempre supera a Nearest Neighbor.

Recomendaciones

- Agregar más casos de prueba, incluyendo grafos no conexos.
- Reconstruir visualmente los caminos mínimos completos entre paradas en la interfaz.
- Permitir cargar archivos JSON desde la interfaz gráfica.
- Exportar reportes de rutas a archivo de texto o PDF.
- Agregar pruebas unitarias automáticas para estructuras y algoritmos.
- Comparar heurísticas contra una solución óptima en instancias pequeñas.

GIT HUB

<https://github.com/IsaacAr12/LogisTEC.git>

l) Bibliografía

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.

Google. (s. f.). Gson User Guide. <https://github.com/google/gson/blob/main/UserGuide.md>

Oracle. (s. f.). Java Platform, Standard Edition Documentation. <https://docs.oracle.com/en/java/>

Oracle. (s. f.). The Java Tutorials: Creating a GUI With Swing. <https://docs.oracle.com/javase/tutorial/uiswing/>

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Sedgewick, R., & Wayne, K. (s. f.). Algorithms, 4th Edition: Graphs. <https://algs4.cs.princeton.edu/40graphs/>